Week 7 Report

Power BI: Students Performance Data Preparation and Modelling

Kamali V

E25DA023

Table of Contents:

Objectives

1. Importing the Data into Power BI
2. Previewing the Students Performance Dataset
3. Standardising Text with Capitalize Each Word
4. Creating the Total Marks Column
5. Calculating the Average Mark
6. Assigning Grades with a Conditional Column
7. Adding the Pass / Fail Result Column
8. Adding Student ID and Reordering Columns
9. Creating the Student Marks Table (Unpivot)
10. Data Model and Measures Table

Objectives:

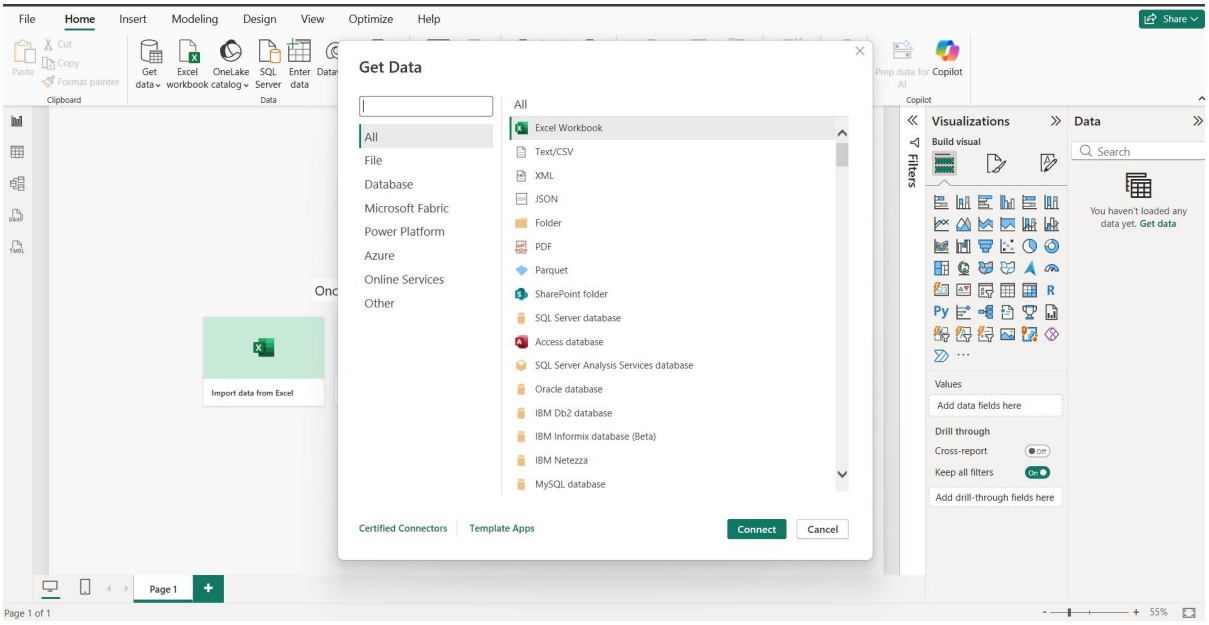
This report documents the Week 7 Power BI work on the Students Performance dataset. The goals of this week were to:

- Import the Students Performance data from a CSV file using Get Data.
- Clean and standardise the text columns in Power Query Editor.
- Create calculated columns: Total Marks, Average Mark, Grade and Result (Pass/Fail).
- Add a Student ID index and organise the column order.
- Build a Student Marks table by unpivoting the subject scores.
- Set up the data model relationships and a Measure Table for reporting.

1. Importing the Data into Power BI:

The Week 7 and 8 work starts from a blank Power BI Desktop report. The Get Data window was opened from the Home ribbon, which lists every connector Power BI supports, such as Excel Workbook, Text/CSV, XML, JSON, Folder, PDF and SQL Server.

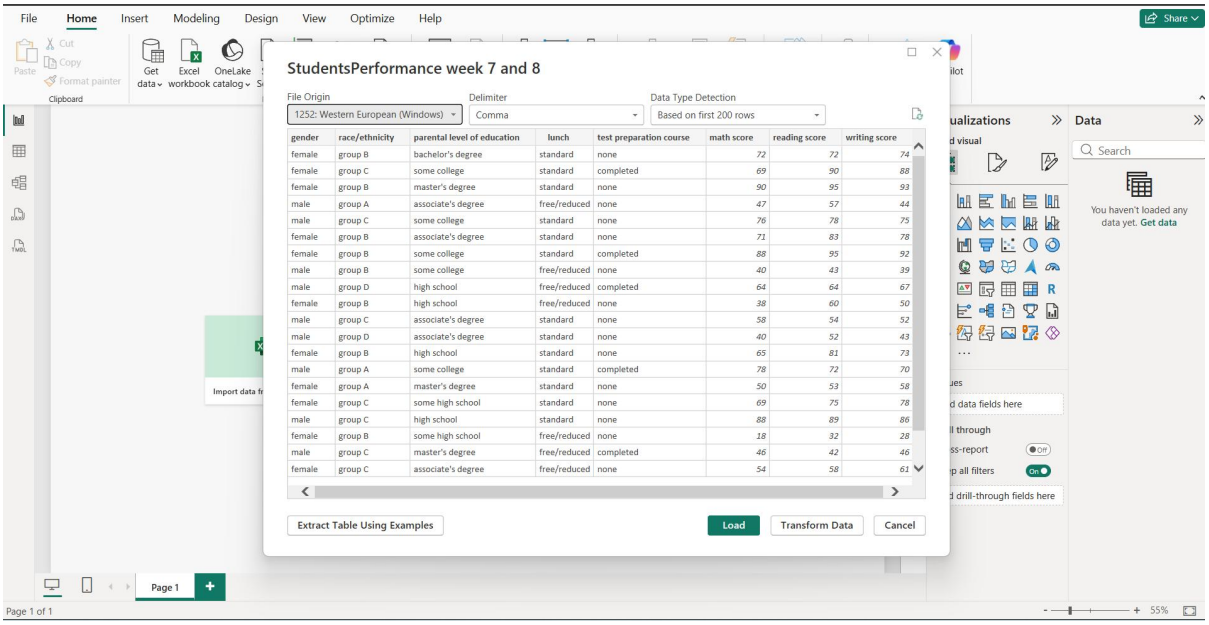
Because the Students Performance dataset is stored as a delimited text file, the Text/CSV connector was used, followed by Connect, to select the file and open the preview window.



2. Previewing the Students Performance Dataset:

After the file was selected, Power BI showed a preview of the StudentsPerformance week 7 and 8 data. The File Origin was Western European (Windows), the Delimiter was Comma and Data Type Detection was set to Based on first 200 rows.

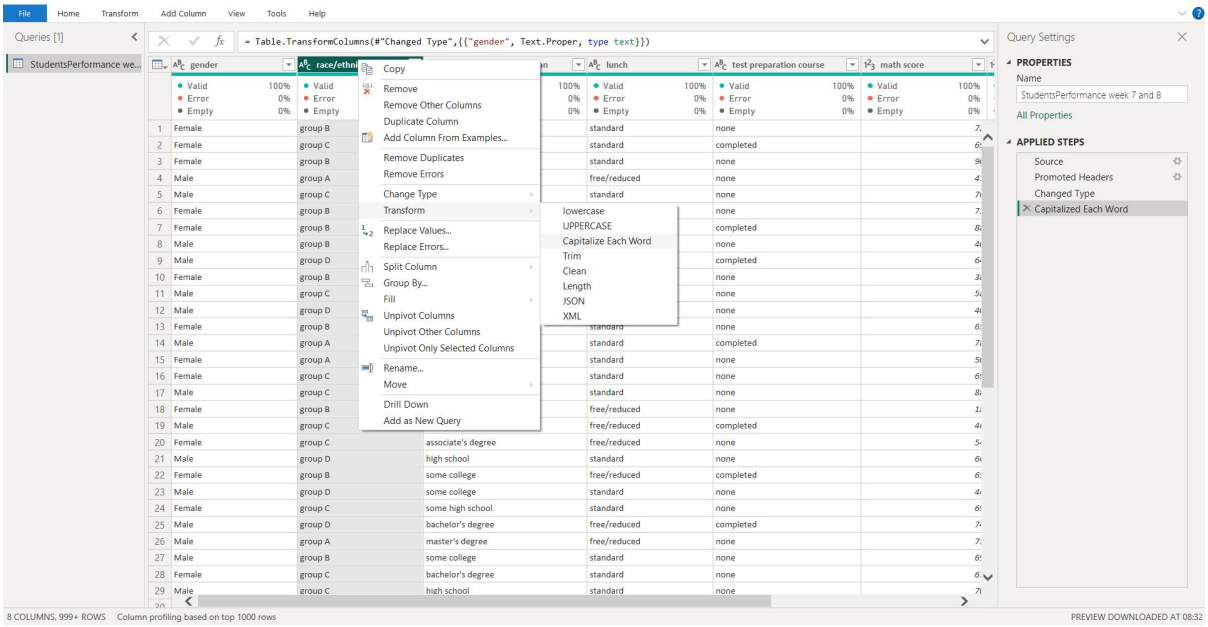
The preview shows the raw columns: gender, race/ethnicity, parental level of education, lunch, test preparation course, math score, reading score and writing score. The values are all in lowercase, so the data needed cleaning before it could be used. Transform Data was chosen to open Power Query Editor.



3. Standardising Text with Capitalize Each Word:

In Power Query Editor the first steps recorded were Source, Promoted Headers and Changed Type. The text columns held lowercase values such as "group b" and "bachelor's degree", so a consistent format was applied.

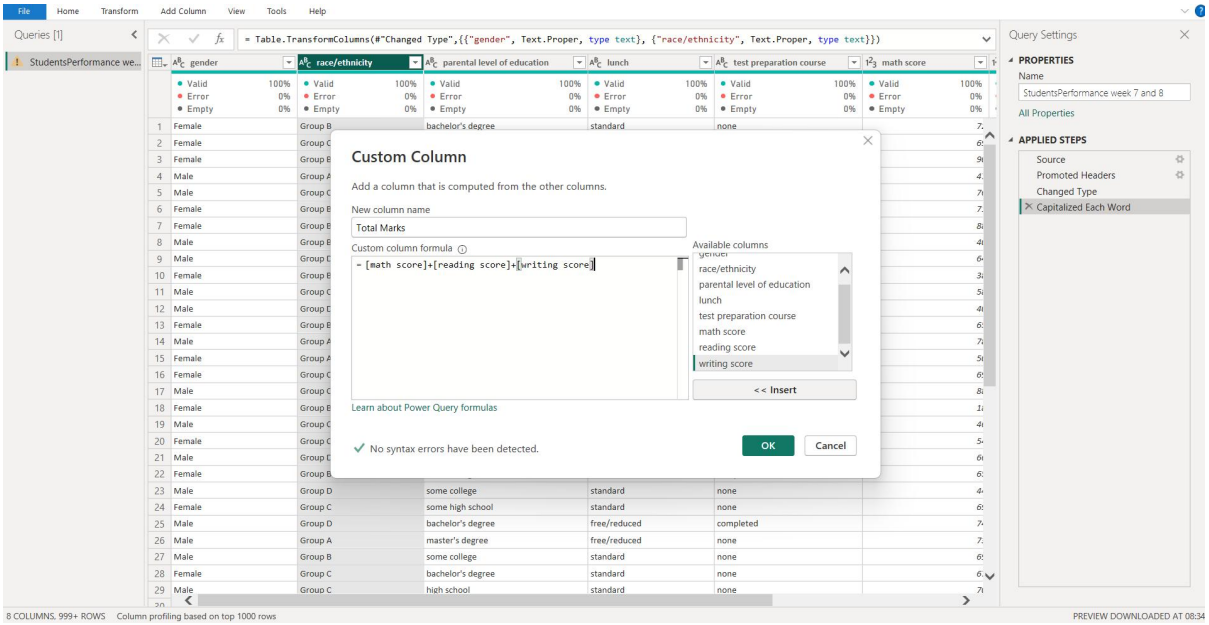
The column was right-clicked and Transform > Capitalize Each Word was selected. The formula bar shows the step as Table.TransformColumns using Text.Proper, which turns "female" into "Female" and "group b" into "Group B". The same transformation was applied to the other text columns.



4. Creating the Total Marks Column:

A custom column was added from the Add Column tab using Custom Column. The new column was named Total Marks and its formula adds the three subject scores together.

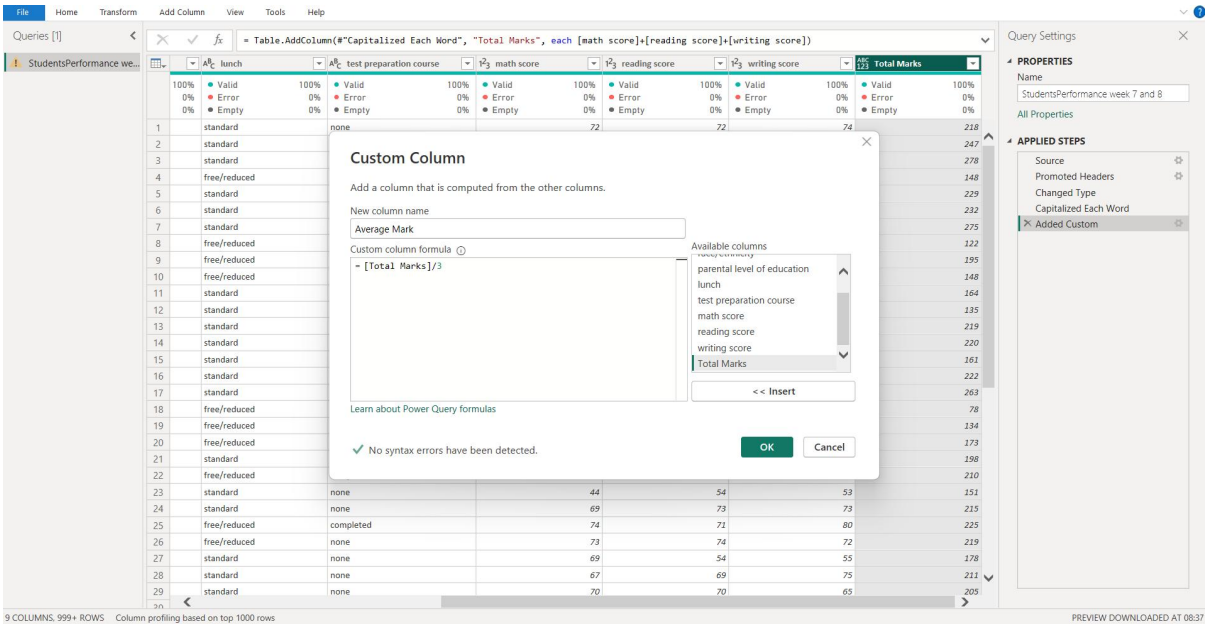
Formula used: = [math score] + [reading score] + [writing score]. Power Query reported that no syntax errors were detected, and the step Added Custom was added to the Applied Steps list.



5. Calculating the Average Mark:

A second custom column, Average Mark, was built on top of Total Marks. It divides the total by 3, the number of subjects.

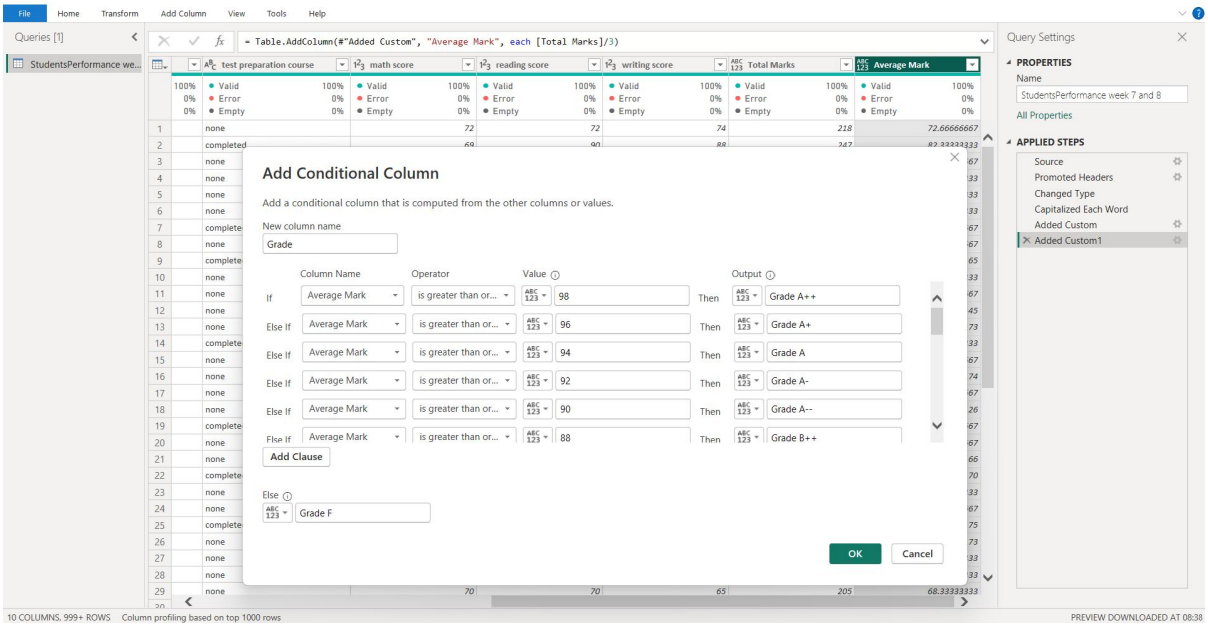
Formula used: = [Total Marks] / 3. The column gives each student a single value (for example 72.67 for the first student) that can be used for grading and for the pass/fail decision.


File
Home
Transform
Add Column
View
Tools
Help
Queries [1]
StudentsPerformance we...
= Table.AddColumn(#"Capitalized Each Word", "Total Marks", each [math score]+[reading score]+[writing score])
lunch
test preparation course
math score
reading score
writing score
Total Marks
Custom Column
Add a column that is computed from the other columns.
New column name
Average Mark
Custom column formula
= [Total Marks]/3
Available columns
parental level of education
lunch
test preparation course
math score
reading score
writing score
Total Marks
<< Insert
Learn about Power Query formulas
No syntax errors have been detected.
OK
Cancel
Query Settings
PROPERTIES
Name
StudentsPerformance week 7 and 8
All Properties
APPLIED STEPS
Source
Promoted Headers
Changed Type
Capitalized Each Word
Added Custom
9 COLUMNS, 999+ ROWS
Column profiling based on top 1000 rows
PREVIEW DOWNLOADED AT 08:37

6. Assigning Grades with a Conditional Column:

The Add Conditional Column dialog was used to create a column named Grade based on Average Mark. Each clause checks "is greater than or equal to" a threshold and returns a grade label, working from the highest band downward.

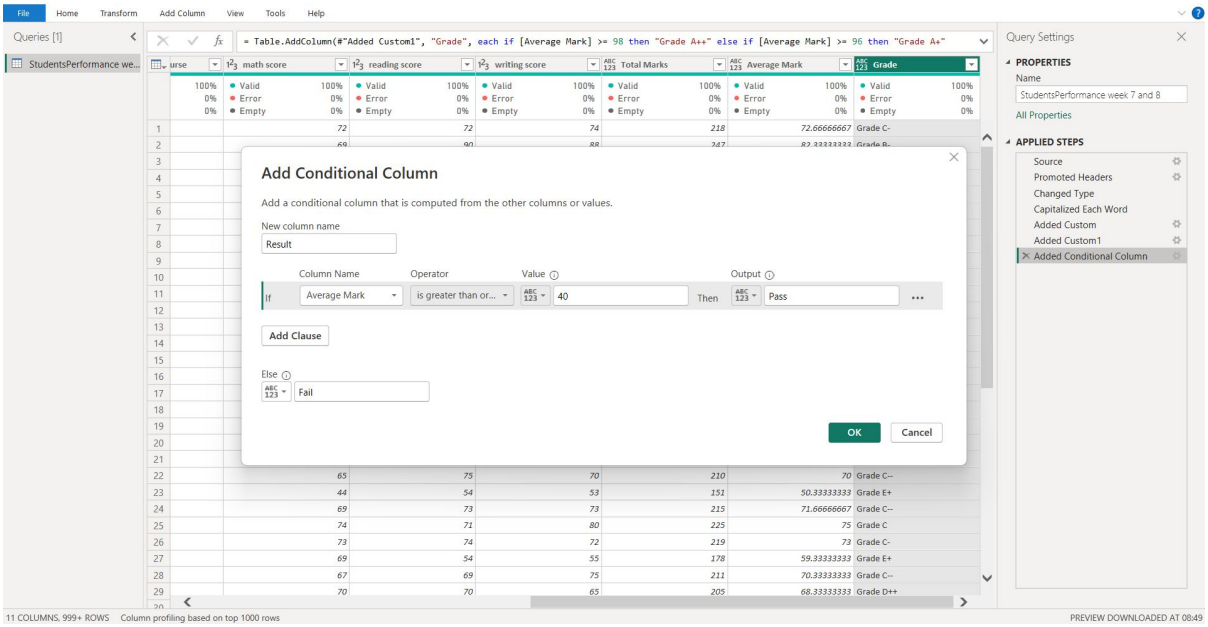
The thresholds visible in the dialog are: 98 = Grade A++, 96 = Grade A+, 94 = Grade A, 92 = Grade A-, 90 = Grade A--, and 88 = Grade B++, with further clauses continuing down the scale. Any value that does not meet a clause falls into the Else value, Grade F.



7. Adding the Pass / Fail Result Column:

Another conditional column, named Result, was added to show whether each student passed. The rule is: if Average Mark is greater than or equal to 40, the output is Pass; otherwise (Else) the output is Fail.

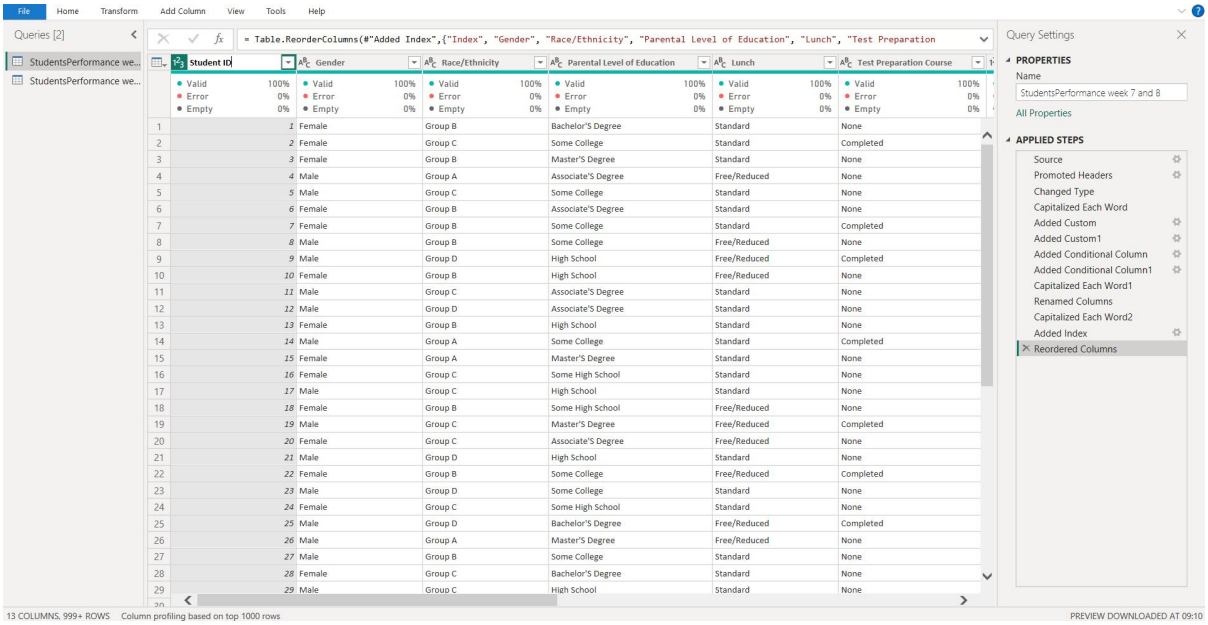
The Grade column created in the previous step is already visible in the data grid, with values such as Grade C-, Grade B- and Grade E+. The Result column is the basis for pass-rate measures in the report.


File
Home
Transform
Add Column
View
Tools
Help
Queries [1]
StudentsPerformance we...
= Table.AddColumn(#"Added Custom1", "Grade", each if [Average Mark] >= 98 then "Grade A++" else if [Average Mark] >= 96 then "Grade A+"
urse
math score
reading score
writing score
Total Marks
Average Mark
Grade
Add Conditional Column
Add a conditional column that is computed from the other columns or values.
New column name
Result
Column Name
Operator
Value
Output
If
Average Mark
is greater than or...
40
Then
Pass
Add Clause
Else
Fail
OK
Cancel
Query Settings
PROPERTIES
Name
StudentsPerformance week 7 and 8
All Properties
APPLIED STEPS
Source
Promoted Headers
Changed Type
Capitalized Each Word
Added Custom
Added Custom1
Added Conditional Column
11 COLUMNS, 999+ ROWS
Column profiling based on top 1000 rows
PREVIEW DOWNLOADED AT 08:49

8. Adding Student ID and Reordering Columns:

To give each student a unique key, an index column was added (step Added Index) and then renamed Student ID. The columns were then renamed in title case (Gender, Race/Ethnicity, Parental Level of Education, Lunch, Test Preparation Course) and rearranged with Reordered Columns so that Student ID comes first.

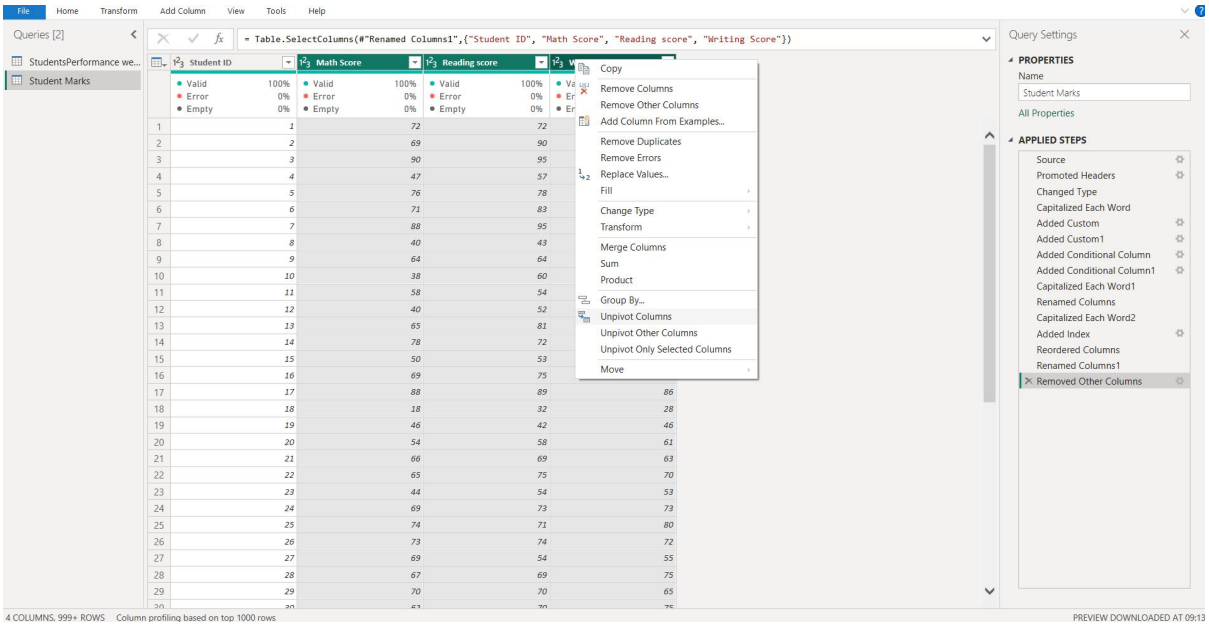
The final column order places the identifier first, followed by the descriptive attributes. The query was named StudentsPerformance week 7 and 8 and now has 13 columns, with 100% valid values and no errors or empty cells.



9. Creating the Student Marks Table (Unpivot):

A second query, Student Marks, was created from the main query. Remove Other Columns kept only Student ID, Math Score, Reading score and Writing Score. The context menu in the screenshot shows the Unpivot Columns options being used on the three score columns.

Unpivoting turns the three score columns into rows with an Attribute column (the subject) and a Value column (the mark). This long format is better suited to analysing marks by subject in charts and DAX measures. Both queries are shown in the Queries pane (Queries [2]).



10. Data Model and Measures Table:

In Model view the tables were connected with relationships on Student ID. Student Details (the descriptive attributes) is linked one-to-one to StudentsPerformance, and StudentsPerformance is linked one-to-many to Student Marks, whose Attribute and Value columns hold the unpivoted subject scores.

A separate Measure Table was created to hold the report calculations: Average Math Score, Average Reading Score, Average Writing Score, Overall Average Score and Overall Pass %. Keeping measures in one table makes them easy to find in the Data pane when building visuals.

